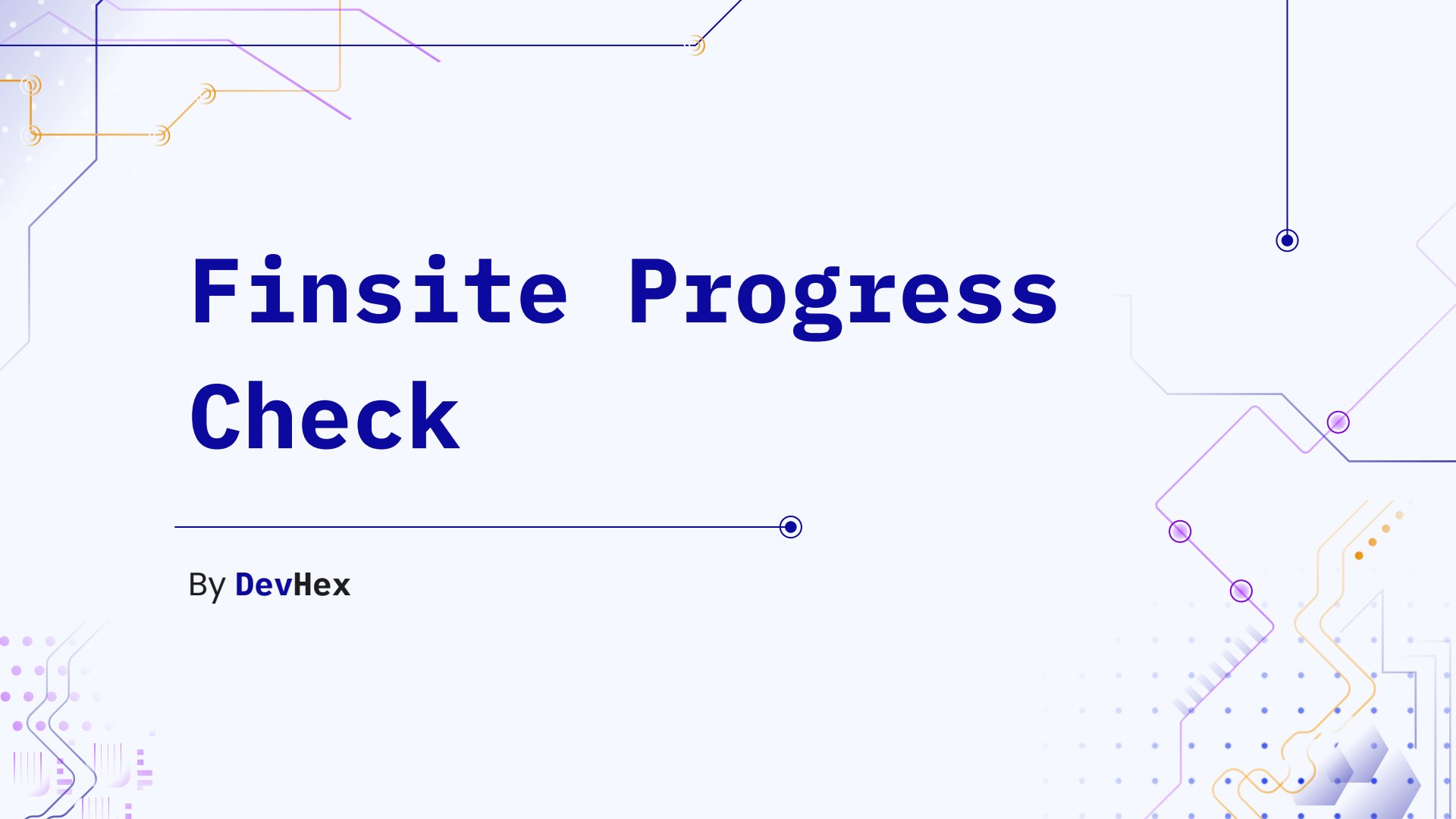




DevHex

The background features a light blue gradient with abstract circuit-like patterns. Purple and orange lines, some straight and some curved, crisscross the frame. Small circles, some solid and some hollow, are placed at various points along these lines. In the bottom right corner, there is a cluster of blue dots and a series of blue arrows pointing towards the right.

Finsite Progress Check

By **DevHex**



01

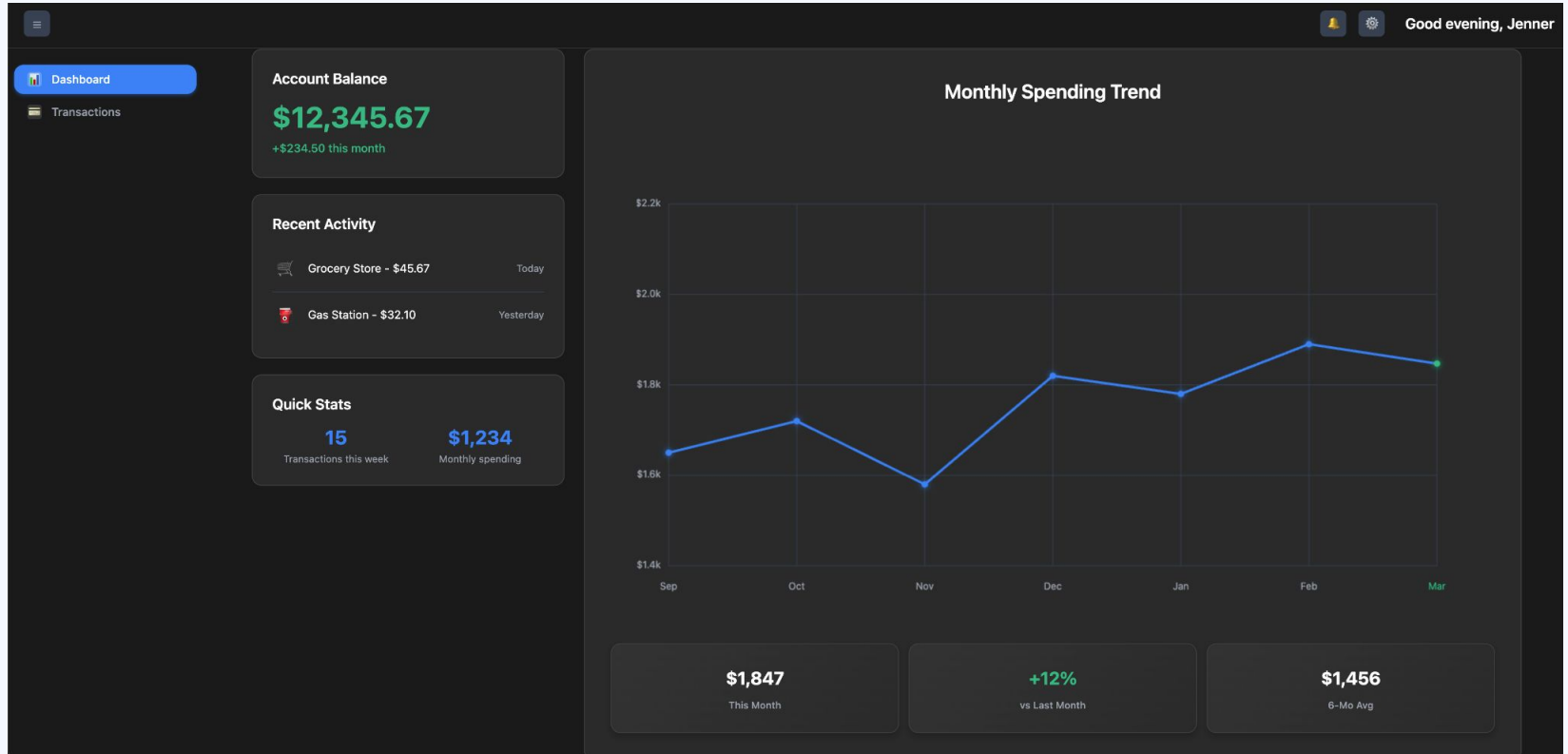
View/UI Adjustments



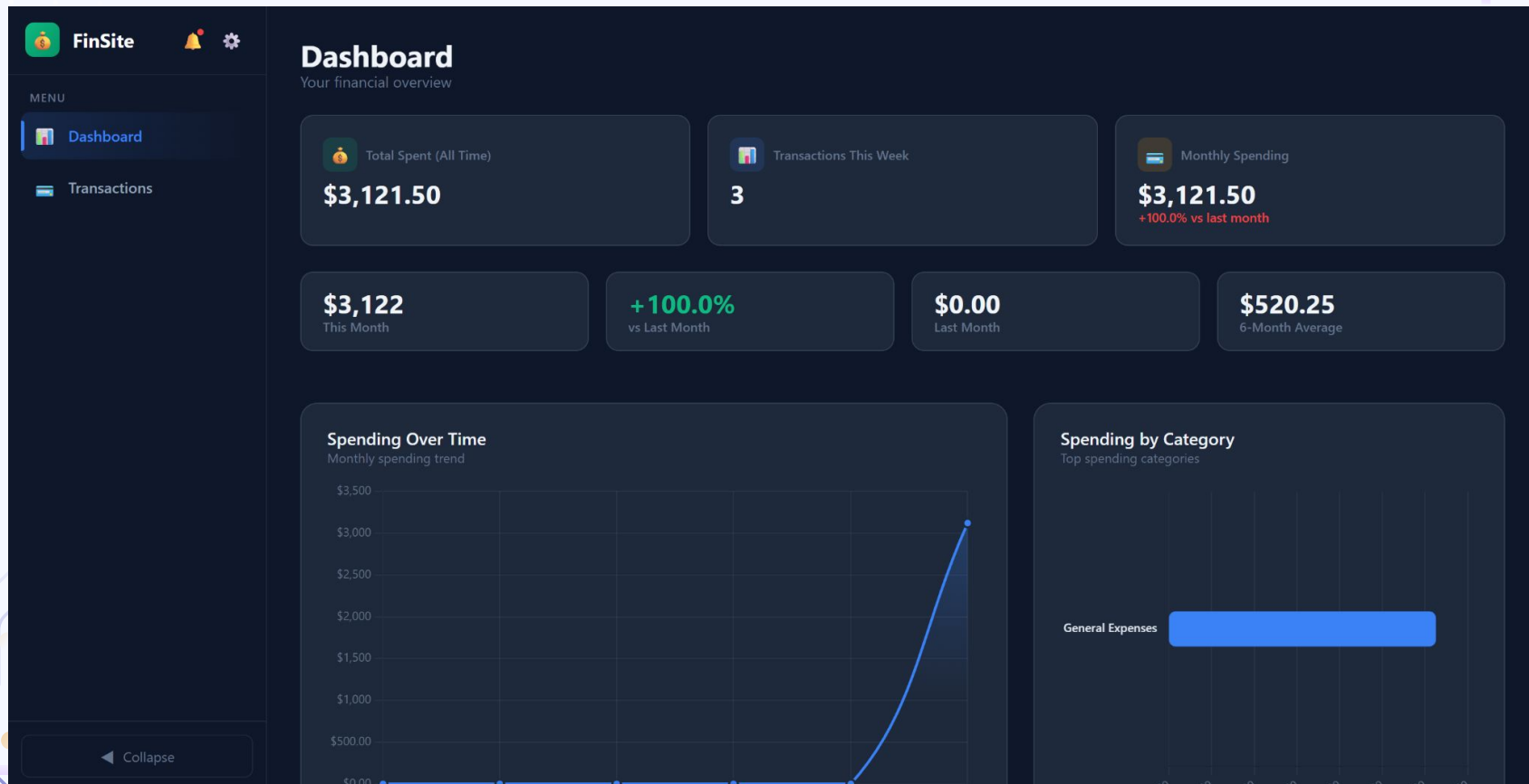
User Interface Updates

- Two pane shell
 - Persistent left sidebar (collapsible)
 - Interchangeable main content area (dashboard, transactions, etc.)
- Dynamic Dashboard
 - Replaced all static dashboard data with real, computed transaction summaries.
 - Model computes all summary metrics (totals, recent, weekly, monthly)
 - Controller updates dashboard summary after any transaction change
 - Dashboard renders only from summary, no calculations in the view

Old Dashboard



New Dashboard



User Interface Updates

- Transactions View
 - Layout: Header, filter bar, and grouped transaction list; light styling with dividers and rounded cards.
 - Actions: Header includes Search, Date, Filters, and Add Transaction; all filters combine and support clear/reset.
 - List: Transactions grouped by date with clear row layout, hover states, sticky header, and empty-state messaging.

Transactions

FinSite

MENU

Dashboard

Transactions

◀ Collapse

Transactions

Search

Date

Filters

+ Add Transaction

All transactions

Edit multiple

Newest first

Today

🖨

GeForce RTX 4090

📄 other

💰 General Expenses

-\$3,000.00

Yesterday

🏌️

Golf

🎬 entertainment

💰 General Expenses

-\$54.00

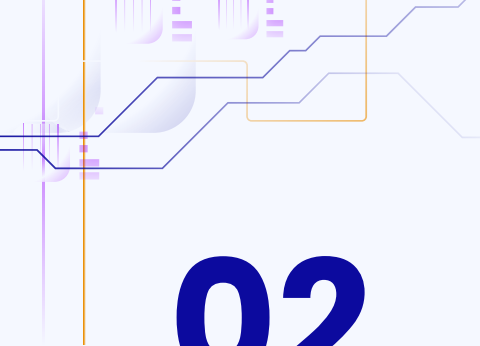
🍜

Ramen Nagi

🍷 dining

💰 General Expenses

-\$67.50



02

ChartJS Implementation



ChartJS Updates

- Expenses over time
 - Displays money spent per month
 - Limit to last N months
 - Consistent amount and date formatting
- Expenses by group
 - Displays aggregated pre-processed totals by category
 - Optionally show top-N groups with “Other” bucket

Charts/Graphs

Spending Over Time

Monthly spending trend



Spending by Category

Top spending categories



ChartJS Integration

- Load ChartJS once and restrict direct use to chart widget
- Adhere to strict data pipeline, passing only aggregated data
- Parent components convert summary data into chart-data objects
- One ChartJS instance per widget to update/destroy as needed
- Update charts once per data change, only when shown
- Pre-aggregate data (monthly buckets, top-N, etc)
- Keep charts lightweight for fast interaction



03

Storage Structure



Updates

- **Defined and documented a transaction object model**
 - Transaction objects had inconsistent structure
 - Now associated with ID, color / icon, appropriate relationships, and other attributes
- **Defined Groups and Category Entities**
 - In the process of replacing hard coded data with data retrieved from the database using distinct objects / payload to dynamically display information
 - Groups: household, investments, expenses
 - Currently have 7 categories across all groups in addition to custom categories
- **Model Layer**
 - Replacing hard coded data and adding structure to the model
 - Working on populating data and data attributes via `init()` after retrieving information from IndexedDB

Updates

- **IndexedDB Schema**

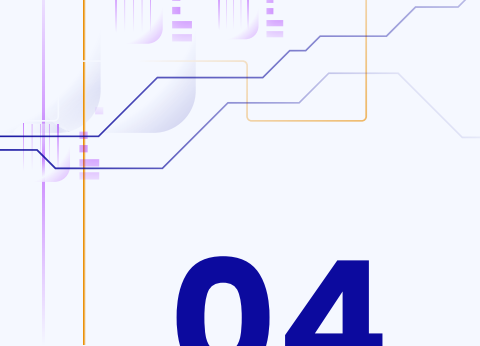
- Unique individual stores allow overall better performance in comparison to 1 store
- **Scalable**: Able to add more stores for other purposes (i.e. budgets or goals)
- **Faster queries**: Focused stores are faster to query than one giant store
- **Clear Domain Model**: Makes code easier to understand and maintain

- **Performance Optimization**

- $O(n) \rightarrow O(1)$
- Recounting every entry vs. Keeping a tab
- Live updates by keeping running totals and updating using individual entries instead of full recalculations everytime.
- Recalculation $O(n)$ happens on `init()`, Bulk import / CSV, Clearing ALL transactions
- Incremental Aggregation: Updating piece by piece instead of recalculating every time

Updates

- **Slow way (Before):**
 - You have a piggy bank with 100 coins
 - Every time someone asks "how much money do you have?", you dump out all 100 coins and count them one by one
 - When you add 1 more coin, you dump out all 101 coins and count again
- **Fast way (After):**
 - You count the 100 coins once at the start: \$50
 - Write "\$50" on a sticky note on the piggy bank
 - When you add a quarter, just update the sticky note: $\$50 + \$0.25 = \$50.25$
 - When someone asks "how much?", just read the sticky note



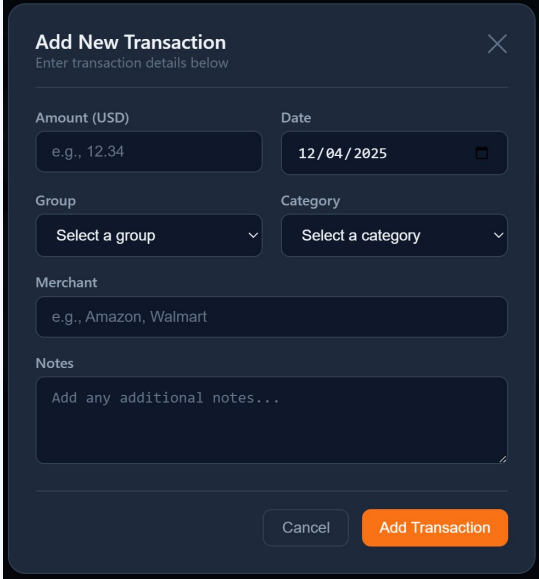
04

Manual Entry



Add Transaction Button

- Manual entry will act as the gateway for any transaction entering the site. Anything parsed or imported will flow into this form, so we added a clear trigger from the Transactions page.
- Component
 - Added primary “Add Transaction” button to `<finsite-transactions>` header (alongside filters/export)
- Behavior
 - Click emits bubbling custom event: `add-transaction`
 - No data handling or view changes inside the component
- Integration
 - Parent can listen and open manual-entry UI in a later refactor



The screenshot shows a dark-themed modal window titled "Add New Transaction" with a close button (X) in the top right corner. Below the title is a subtitle "Enter transaction details below". The form contains several input fields: "Amount (USD)" with a placeholder "e.g., 12.34", "Date" with a placeholder "12/04/2025" and a calendar icon, "Group" with a dropdown menu showing "Select a group", and "Category" with a dropdown menu showing "Select a category". Below these is a "Merchant" field with a placeholder "e.g., Amazon, Walmart". At the bottom is a "Notes" section with a text area containing the placeholder "Add any additional notes...". At the very bottom of the modal are two buttons: "Cancel" and "Add Transaction".

Merchant Field Support

- We added a dedicated merchant field and kept notes for comments only to simplify parsing and imports. Separating merchant from notes lets future CSV, receipt, or email parsing map directly to a merchant without scraping free-text.
- Model + Storage
 - Added `merchant` string to transaction schema
 - CRUD persists/returns `merchant`
- UI
 - Manual-entry form: dedicated merchant input next to group/category
 - Notes remain comment-only
- Compatibility
 - Existing transactions remain valid; merchant defaults empty

Q&A



Thanks !